

Formal Methods for System Design

Chapter 5: Symbolic model checking

Mickael Randour

LoFI — Logic and Foundations of Intelligent Systems

Mathematics Department, Faculty of Science, UMONS

October 2026

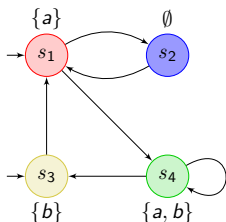


- 1 Symbolic model checking: why, what and how?
- 2 CTL model checking through switching functions
- 3 Efficient encoding through ROBDDs
- 4 A glance at other approaches for efficient model checking

7 / 67

Binary encoding of transitions

Example



■ $|S| = 4 \implies n = \log |S| = 2$.

■ We choose an encoding $enc: S \rightarrow \{0, 1\}^2$:

$$enc(s_1) = 00$$

$$enc(s_2) = 01$$

$$enc(s_3) = 10$$

$$enc(s_4) = 11$$

Same idea for *transitions*: $\rightarrow \subseteq S \times S$ is represented by a Boolean function $\Delta: \{0, 1\}^{2n} \rightarrow \{0, 1\}$ assigning 1 to pairs of bit vectors $(\bar{b}, \bar{b}')$ such that $s = enc^{-1}(\bar{b})$, $s' = enc^{-1}(\bar{b}')$ and $s \rightarrow s'$.

In the following, we discuss how this encoding can be seen as switching functions and how CTL model checking can be formulated on them. Then we present compact representation of switching functions through ROBDDs.

Adaptation to $\exists \square B$

We need to mimic $T_0 = B$ and

$T_{j+1} = T_j \cap \{s \in S \mid \exists s' \in S, s' \in Post(s) \wedge s' \in T_j\}$ by a symbolic computation.

Input: $\Delta(\bar{x}, \bar{x}')$ and χ_B

Output: $f_i(\bar{x})$ representing $Sat(\exists \Box B)$

$$f_0(\overline{X}) := \chi_B(\overline{X})$$
$$j := 0$$

repeat

$$f_{j+1}(\bar{x}) := f_j(\bar{x}) \wedge \exists \bar{x}'. (\Delta(\bar{x}, \bar{x}') \wedge f_j(\bar{x}'))$$
$$j := j + 1$$

until $f_j(\bar{x}) = f_{j-1}(\bar{x})$

```

return  $f_j(\bar{x})$ 

```

⇒ Blackboard illustration on last example for $\exists \square b$.

Wrap-up

We now have **all the necessary ingredients to deal with CTL formulae in ENF symbolically**: algorithms for atomic propositions, conjunctions, negations, $\exists \bigcirc \Phi$, $\exists(\Phi \cup \Psi)$ and $\exists \square \Phi$ based on switching functions.

E.g., $Sat(\Phi \wedge \neg\Psi)$ is given by $f_\Phi \wedge \neg f_\Psi$.

At the end of the model checking process, we must check that $I \subseteq \text{Sat}(\Phi)$: this can be achieved by checking that $\chi_I \rightarrow f_\Phi$ holds, i.e., that $\neg \chi_I \vee f_\Phi = 1$ in terms of switching functions.

⇒ It remains to find appropriate data structures to encode the switching functions!

- ### 3 Efficient encoding through ROBDDs

The problem

We are looking for an appropriate data structure to encode the switching functions. It needs to:

- 1 yield **compact representations** of the satisfaction sets and the transition relation,
- 2 **support Boolean connectives and comparison of switching functions** (to implement the model checking procedures based on switching functions directly).

Regarding compactness, recall that for a system with $|S| = 2^n$ states, we only need n Boolean variables ($2n$ for transitions).

⇒ Unfortunately, it can be proved that *no* data structure can ensure polynomial-size representations of *all* switching functions (because there are doubly-exponentially-many switching functions for m variables).

Possible data structures

We know that no structure can avoid exponential representation for *some* switching functions.

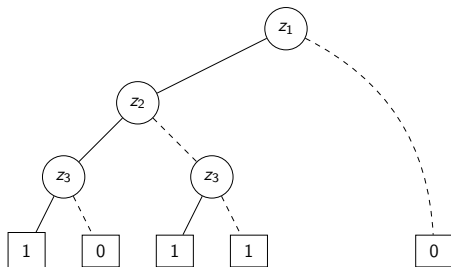
⇒ **Does not mean that all structures are equally good!**

- *Truth tables* are not efficient as they *always* require 2^m entries for m variables (i.e., whatever the switching function).
- Same for *binary decision trees*: always $2^{m+1} - 1$ nodes.
- *Conjunctive and disjunctive normal forms* are not well-suited because equivalence checking is hard.
- A good choice is **ordered binary decision diagrams (OBDDs)**.
 - ▷ Yields compact representations *for many switching functions appearing in practical applications*.
 - ▷ Boolean connectives in linear time (in the input OBDDs).
 - ▷ Equivalence checking in constant time with appropriate implementation.

OBDDs

Intuition

Main idea: compactification of binary decision trees.



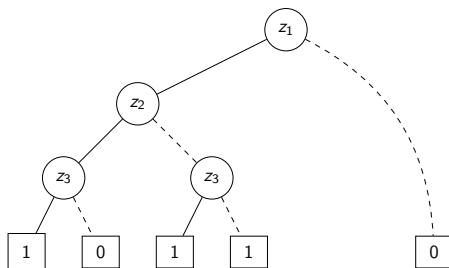
The right subtree corresponds to cofactor $f|_{z_1=0}$, modeling the constant function 0: tests on z_2 and z_3 are useless as all terminal nodes have value 0.

⇒ We replace this subtree by a single terminal node of value 0.

OBDDs

Intuition

Main idea: compactification of binary decision trees.

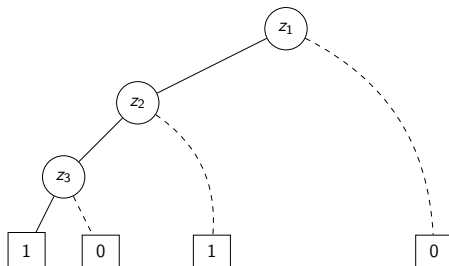


The subtree corresponding to cofactor $f|_{z_1=1, z_2=0}$ is also constant.

OBDDs

Intuition

Main idea: compactification of binary decision trees.



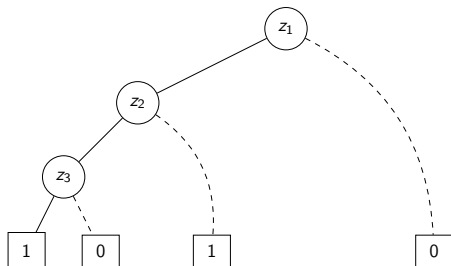
The subtree corresponding to cofactor $f|_{z_1=1, z_2=0}$ is also constant.

⇒ We replace this subtree by a single leaf of value 1.

OBDDs

Intuition

Main idea: compactification of binary decision trees.

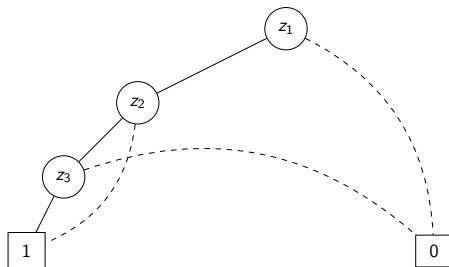


Now, we finally observe that it is **useless to keep several leaves with the same value**.

OBDDs

Intuition

Main idea: compactification of binary decision trees.



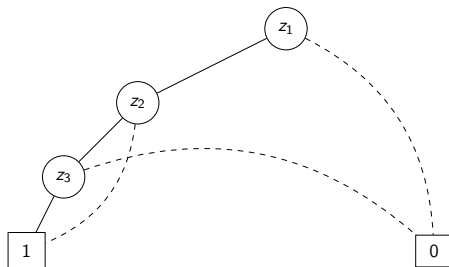
Now, we finally observe that it is **useless to keep several leaves with the same value.**

⇒ **We only keep one of each.**

OBDDs

Intuition

Main idea: compactification of binary decision trees.



At the end, we obtain a *directed acyclic graph (DAG)* where inner nodes have outdegree 2. As in BDTs, inner nodes are labeled with variables and outgoing edges (solid or dashed) correspond to their evaluations; leaves are labeled with the function value.

⇒ **Much more compact.**

OBDDs

Other example

⇒ **Blackboard example for $f = (z_1 \wedge z_3) \vee (z_2 \wedge z_3)$.**

OBDDs

Variable ordering

Definition: variable ordering

Let Var be a finite set of variables. A *variable ordering* for Var is any tuple $\varphi = (z_1, \dots, z_m)$ such that $Var = \{z_1, \dots, z_m\}$ and $z_i \neq z_j$ for $1 \leq i < j \leq m$.

$\hookrightarrow$ It induces a total order and operators $<_{\varphi}$ and $\leq_{\varphi}$:
i.e., $z_i <_{\varphi} z_j$ iff $i < j$.

$\Rightarrow$ **We will see that different orderings yield different (R)OBDDs!**

OBDDs

Definition (2/2)

The variable labeling function must be **consistent with the ordering**: if v is an inner node and w is both a successor of v and an inner node, then $var(v) < var(w)$ must hold.

↪ Intuitively, branches must respect the variable ordering.

When referring to the *size* of an OBDD, we consider its number of nodes.

⚠ **Observe that the definition of OBDDs does not enforce the reducing operations we have discussed before. In particular, BDTs are valid OBDDs.**

OBDDs

Semantics

As observed intuitively, the semantics of a $\wp$ -OBDD $\mathfrak{B}$ is the **switching function** $f_{\mathfrak{B}}$ for Var where $f_{\mathfrak{B}}([z_1 = b_1, \dots, z_m = b_m])$ is the value of the terminal node reached by following the corresponding branch of $\mathfrak{B}$ from the root v_0 .

⇒ In the following, we will see how to go from OBDDs to Reduced OBDDs (ROBDDs): for that, we need to introduce a few more concepts.

OBDDs

Bottom-up characterization of switching functions for nodes

Let $\mathfrak{B}$ be a $\wp$ -OBDD. The switching functions f_v for the nodes $v \in V$ are given as follows:

- if v is a leaf, then f_v is the constant switching function with value $val(v)$;
- if v is a z -node, then $f_v = (\neg z \wedge f_{succ_0(v)}) \vee (z \wedge f_{succ_1(v)})$.

Furthermore, $f_{\mathfrak{B}} = f_{v_0}$ for the root v_0 of $\mathfrak{B}$.

$\implies$ **Observe the Shannon expansion!**

$\hookrightarrow$ All concepts of OBDD-based approaches are based on this decomposition into cofactors.

OBDDs

$\wp$ -consistent cofactors (1/2)

Definition: $\wp$ -consistent cofactor

Let f be a switching function for Var and $\wp = (z_1, \dots, z_m)$ be an ordering for Var . A switching function f' for Var is a **$\wp$ -consistent cofactor** of f if there exists $i \in \{0, \dots, m\}$ such that

$$f' = f|_{z_1=b_1, \dots, z_i=b_i}.$$

E.g., let $f = z_1 \wedge (z_2 \vee \neg z_3)$ and $\wp = (z_1, z_2, z_3)$. Consistent cofactors are:

- ▷ f itself;
- ▷ $f|_{z_1=0} = 0$ and $f|_{z_1=1} = z_2 \vee \neg z_3$;
- ▷ $f|_{z_1=1, z_2=0} = \neg z_3$ and $f|_{z_1=1, z_2=1} = 1$;
- ▷ $f|_{z_1=1, z_2=0, z_3=0} = 1$ and $f|_{z_1=1, z_2=0, z_3=1} = 0$ (redundant).

$\implies$ **$\wp$ -consistent cofactors: f , $z_2 \vee \neg z_3$, $\neg z_3$, 0 and 1.**

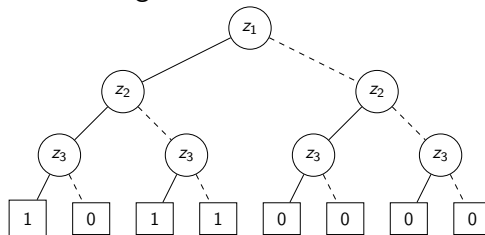
OBDDs

$\wp$ -consistent cofactors (2/2)

Observation

For each node v in a $\wp$ -OBDD $\mathfrak{B}$, the switching function f_v is a $\wp$ -consistent cofactor of $f_{\mathfrak{B}}$; and for each $\wp$ -consistent cofactor f' of f , there is *at least* one node v in $\mathfrak{B}$ such that $f_v = f'$.

$\implies$ At least one, but **there can be many more!** E.g., BDTs bearing redundant information.



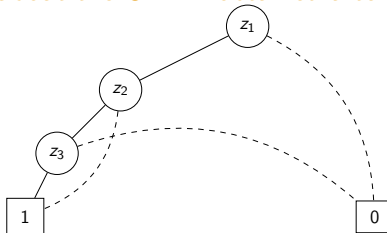
OBDDs

$\wp$ -consistent cofactors (2/2)

Observation

For each node v in a $\wp$ -OBDD $\mathfrak{B}$, the switching function f_v is a $\wp$ -consistent cofactor of $f_{\mathfrak{B}}$; and for each $\wp$ -consistent cofactor f' of f , there is *at least* one node v in $\mathfrak{B}$ such that $f_v = f'$.

⇒ What about the OBDD obtained after “reduction”?



⇒ It is free of redundancies: every $\wp$ -consistent cofactor is represented by a single node.

Reduced OBDDs

Definition

Definition: reduced OBDD (ROBDD)

Let $\mathfrak{B}$ be a $\wp$ -OBDD. It is said to be *reduced* if for every pair of nodes (v, w) in $\mathfrak{B}$:

$$v \neq w \implies f_v \neq f_w.$$

The fact that each $\wp$ -consistent cofactor corresponds to exactly one node of a $\wp$ -ROBDD is the crux to obtain the next theorem.

Reduced OBDDs

Universality and canonicity

Theorem: universality and canonicity of ROBDDs

Let Var be a finite set of variables and $\wp$ an ordering for Var .

Then:

- (a) for each switching function f for Var , there exists a $\wp$ -ROBDD $\mathfrak{B}$ with $f_{\mathfrak{B}} = f$;
- (b) given two $\wp$ -ROBDDs $\mathfrak{B}$ and $\mathfrak{C}$ with $f_{\mathfrak{B}} = f_{\mathfrak{C}}$, then $\mathfrak{B}$ and $\mathfrak{C}$ are isomorphic, i.e., they agree up to renaming of the nodes.

$\implies$ It is possible to talk about “the $\wp$ -ROBDD” for a switching function f (up to isomorphism): this ROBDD is also the minimal $\wp$ -OBDD for f .

Reduced OBDDs

Simple construction procedure based on consistent cofactors

Let f be the switching function for Var to represent and $\wp$ the ordering of the variables.

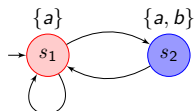
- If f is constant, then the ROBDD $\mathfrak{B}$ contains a single terminal node of the corresponding value. Else we proceed as follows.
- Let V be the set of $\wp$ -consistent cofactors of f .
 - ▷ The root of $\mathfrak{B}$ is f and the constant cofactors are the leaves with corresponding values.
 - ▷ For $f' \in V \setminus \{0, 1\}$, let $var(f') = \min\{z \in Var \mid z \text{ is essential for } f'\}$ (resp. to the ordering).
 - ▷ Successors functions are $succ_0(f') = f'|_{z=0}$ and $succ_1(f') = f'|_{z=1}$ where $z = var(f')$.

By construction, this yields a $\wp$ -OBDD $\mathfrak{B}$, and by Shannon expansion, its semantics is indeed $f_{\mathfrak{B}} = f$ and it is reduced (as each node represents a different cofactor).

Reduced OBDDs

Example

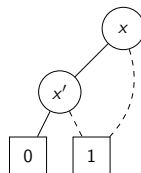
By **universality of ROBDDs**, any switching function can be encoded: let us encode the transition relation of the previous TS.



Need a single Boolean variable x for the encoding:
 $enc(s_1) = 0, enc(s_2) = 1$.

► Transitions: $\Delta = \neg x \vee \neg x'$.

⇒ **Blackboard construction.**



ROBDD $\wp = (x, x')$.

Reduced OBDDs

Consequences of canonicity

The canonicity of ROBDDs yields interesting properties.

- *Absence of redundant vertices*: if $f_{\mathfrak{B}}$ does not depend on x_i , then $\mathfrak{B}$ does not contain an x_i -node.
- *Test for equivalence* between two switching functions $f(\bar{x})$ and $f'(\bar{x})$ can be done by generating ROBDDs $\mathfrak{B}_f$ and $\mathfrak{B}_{f'}$ and checking their isomorphism.
- *Test for validity*: checking if $f(\bar{x}) = 1$ can be done by generating $\mathfrak{B}_f$ and checking that it only consists of a 1-leaf.
- *Test for implication*: checking if $f(\bar{x}) \rightarrow f'(\bar{x})$ by generating $\mathfrak{B}_{f \wedge \neg f'}$ and checking that it only consists of a 0-leaf.
- *Test for satisfiability* of a Boolean expression f by checking that $\mathfrak{B}_f$ contains a reachable 1-leaf.

⚠ The SAT problem is NP-complete! Further proof that ROBDDs cannot ensure polynomial encoding of all switching functions.

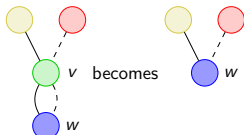
Reduced OBDDs

From OBDDs to ROBDDs (1/2)

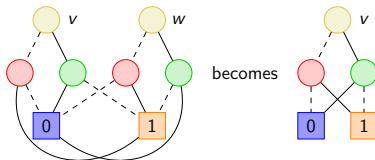
Reduction rules

Any $\wp$ -OBDD can be transformed into a canonical $\wp$ -ROBDD for the same switching function by successive applications of two simple *local reduction rules*.

- 1 *Elimination rule*: if $v \in V_I$ is s.t. $\text{succ}_0(v) = \text{succ}_1(v) = w$, then remove v and redirect all its incoming edges to w .
- 2 *Isomorphism rule*: if $v \neq w$ are the roots of isomorphic trees, remove w and redirect all its incoming edges to v .



Elimination rule.



Isomorphism rule.

Reduced OBDDs

From OBDDs to ROBDDs (2/2)

This reduction scheme is

- **sound**: if $\mathcal{C}$ is a $\wp$ -OBDD obtained by reduction from $\mathfrak{B}$, then $f_{\mathcal{C}} = f_{\mathfrak{B}}$;
- **complete**: the $\wp$ -OBDD $\mathfrak{B}$ is a $\wp$ -ROBDD iff no reduction rule can be applied to $\mathfrak{B}$.

It can be implemented in *linear time* in the size of $\mathfrak{B}$.

Reduced OBDDs

The variable ordering problem

ROBDDs are canonical... **for a fixed variable ordering!**

⇒ The size of the ROBDD *crucially depends on the ordering*
(recall that $|V| = \#$ of $\wp$ -consistent cofactors of f).

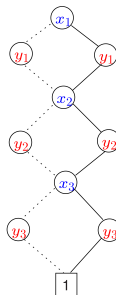
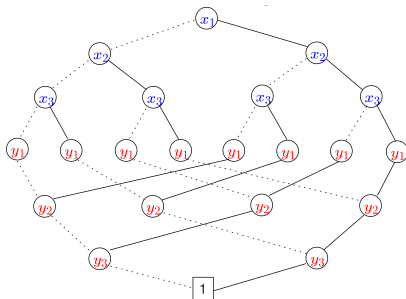
Some functions have

- *both linear and exponential* ROBDDs depending on the chosen ordering: e.g., the *stable function*;
↪ See next slide.
- *only polynomial* ROBDDs: e.g., *symmetric functions* such as $f(\bar{x}) = x_1 \oplus \dots \oplus x_n$ or $f(\bar{x}) = 1$ iff $\geq k$ variables x_i are true;
- *only exponential* ROBDDs: e.g., the middle bit of the multiplication function.

↪ See book.

Reduced OBDDs

The stable function: exponential vs. linear ROBDDs



Example from [Kat10]: $f_{stab}(\bar{x}, \bar{y}) = (x_1 \leftrightarrow y_1) \wedge \dots \wedge (x_n \leftrightarrow y_n)$.

- ▷ $\wp = (x_1, \dots, x_n, y_1, \dots, y_n)$ yields $\mathcal{O}(2^n)$ nodes.
- ▷ $\wp = (x_1, y_1, \dots, x_n, y_n)$ yields $\mathcal{O}(n)$ nodes.

⇒ Intuitively, the second ordering checks each conjunct sequentially whereas the first one needs to recall the values of all variables x_i before being able to check the first conjunct.

Reduced OBDDs

Finding a good ordering

- The size of ROBDDs drastically depends on the ordering.
- Can we determine which is the best ordering, i.e., yielding the minimal ROBDD?
 - ▷ **Not efficiently**: checking if a variable ordering is optimal is NP-hard.

⇒ In practice, efficient heuristics are used to improve the current ordering and rearrange the ROBDD. Beyond the scope of this course.

⇒ For transition relations, the **interleaved ordering** usually yields compact ROBDDs:

for $\Delta(\bar{x}, \bar{x}')$, use $\wp = (x_1, x'_1, \dots, x_n, x'_n)$.

Back to CTL model checking

We already established

- 1 a symbolic CTL model checking procedure based on *switching functions*,
- 2 that switching functions can be represented by *ROBDDs* which, for practical cases, are often compact.

The missing piece is how to *actually* implement the model checking blocks based on ROBDD-representations of the switching functions.

⇒ We need to be able to **synthesize an ROBDD for a switching function f** (as sketched before), but also to **implement Boolean connectives at the ROBDD level**.

I.e., given $\wp$ -ROBDDs for f_1 and f_2 , we must be able to build a $\wp$ -ROBDD for $f_1 \text{ op } f_2$ where op is a Boolean connective (conjunction, implication, etc).

Synthesis of ROBDDs

We concentrate on the problem of synthesizing $\mathfrak{B}_{f_1 \text{ op } f_2}$ from $\mathfrak{B}_{f_1}$ and $\mathfrak{B}_{f_2}$.

⇒ We do not address the problem in full details but sketch the key steps of an approach based on **shared OBDDs**.

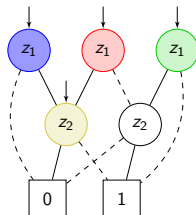
- ▷ The idea is to use a *single* ROBDD with global variable ordering $\wp$ to represent *several* switching functions.
- ▷ The shared OBDD can be seen as the combination of several ROBDDs obtained by *sharing nodes for common $\wp$ -consistent cofactors*.

⇒ **Increased compactness**: in the worst case, the shared OBDD will have its size bounded by the sum of sizes for all combined ROBDDs, but in practice it is often much more compact as shared cofactors are common.

Shared OBDDs

Definition: shared OBDD (SOBDD)

A $\wp$ -SOBDD is simply a $\wp$ -ROBDD with *possibly multiple roots* instead of a single one.



SOBDD representing $f_1 = z_1 \wedge \neg z_2$, $f_2 = \neg z_2$, $f_3 = z_1 \oplus z_2$ and $f_4 = \neg z_1 \vee z_2$ for ordering $\wp = (z_1, z_2)$.

Using SOBDDs for model checking a CTL formula Φ

Sketch (1/2)

We use a *single SOBDD* to encode:

- $\Delta(\bar{x}, \bar{x}')$ for the transition function,
- functions $(f_a)_{a \in AP}$ for the satisfaction sets $Sat(a)$ where $a \in AP$,
- the satisfaction sets $Sat(\Psi)$ for the subformulae Ψ of Φ .

In practice, the *interleaved ordering* gives good results.

Using SOBDDs for model checking a CTL formula Φ

Sketch (2/2)

Model checking process:

- 1 At start, we synthesize an SOBDD representing Δ and functions $(f_a)_{a \in AP}$.
- 2 During the procedure, we *extend it with new root nodes* for characteristic functions $\chi_{Sat(\Psi)}$ for subformulae Ψ of Φ .
 - ▷ E.g., if $\Phi = a \wedge \neg b$, then we first have to insert a root for $f_{\neg b} = \neg f_b$, and then a root for $f_a \wedge f_{\neg b}$.
 - ▷ For formulae like $\exists \square \Psi$, we need to compute a sequence of iterations f_i , and each of them must be added to the SOBDD.
 - ▷ Each root addition *may induce the addition of consistent cofactors* not already present in the SOBDD.

Operations on the SOBDD are interleaved with reduction rules to ensure redundancy-freedom at any time, making the comparison between two functions easy (it boils down to node equality).

Synthesizing SOBDDs

Two tables: *unique* and *computed*

The synthesis process relies on **two tables** for its computations.

■ The **unique** table.

- ▷ Keeps track of created nodes.
- ▷ Each inner node v has an entry $\langle \text{var}(v), \text{succ}_1(v), \text{succ}_0(v) \rangle$.
- ▷ Access via **find_or_add**(z, v_1, v_0) with $v_1 \neq v_0$:
 - returns v if there is a node $v = \langle z, v_1, v_0 \rangle$ in the SOBDD,
⇒ **Isomorphism reduction rule.**
 - if not, creates a new z -node v with $\text{succ}_1(v) = v_1$ and $\text{succ}_0(v) = v_0$.
- ▷ Implemented via *hash functions* (expected access in $\mathcal{O}(1)$).

■ The **computed** table.

- ▷ Keeps track of already computed tuples for upcoming function ITE (*memoization*).
- ▷ Avoids redundant computations.

Synthesizing SOBDDs

The ITE operator (1/3)

We deal with *all* Boolean connectives through a *single ternary operator*, *ITE* (if-then-else):

$$\text{ITE}(g, f_1, f_2) = (g \wedge f_1) \vee (\neg g \wedge f_2).$$

$\implies$ If g then f_1 else f_2 .

Link with the *unique* table representation of node v :

$$f_v = \text{ITE}(z, f_{\text{succ}_1(v)}, f_{\text{succ}_0(v)}).$$

$\implies$ Again the Shannon expansion!

Synthesizing SOBDDs

The ITE operator (2/3)

The $\text{ITE}(g, f_1, f_2) = (g \wedge f_1) \vee (\neg g \wedge f_2)$ operator can give us:

$$f_1 = \text{ITE}(1, f_1, f_2) \quad f_2 = \text{ITE}(0, f_1, f_2) \quad \neg f = \text{ITE}(f, 0, 1)$$

$$f_1 \vee f_2 = \text{ITE}(f_1, 1, f_2) \quad f_1 \wedge f_2 = \text{ITE}(f_1, f_2, 0) \quad f_1 \rightarrow f_2 = \text{ITE}(f_1, f_2, 1)$$

$$f_1 \oplus f_2 = \text{ITE}(f_1, \neg f_2, f_2) = \text{ITE}(f_1, \text{ITE}(f_2, 0, 1), f_2).$$

Key observation

Let g, f_1, f_2 be switching functions for Var , $z \in \text{Var}$ and $b \in \{0, 1\}$. Then:

$$\text{ITE}(g, f_1, f_2)|_{z=b} = \text{ITE}(g|_{z=b}, f_1|_{z=b}, f_2|_{z=b}).$$

⇒ This observation is at the basis of the upcoming algorithm.

The BDD-based approach

- does not reduce the *worst-case* complexity of CTL model checking;
- *greatly helps in many practical applications.*
 - ▷ Depending on the underlying structure of the TS, gains can be huge.
 - ▷ In some applications, TSs with 10^{120} states could be verified through this technique [Kat10] (and could not by explicit techniques).

- 63 / 67

- ▶ not only for CTL,
- ▶ not only via symbolic techniques.

⇒ We briefly mention some of them in the next slides
(non-exhaustive list).

Consider a TS arising from the interleaving between different processes and an LTL or CTL* formula to check.

- ⇒ Partial order reduction techniques aim at reducing the state space by reducing the # of orderings to consider.

⇒ Can lead to huge gain since the state space grows exponentially with the # of processes.

- No guarantee that the system will work as intended beyond K steps.

- In practice, **most errors can be detected** after a reasonable number of steps.

- Technically based on extremely optimized SAT solvers.

- ▷ **NP-hard problem** but efficiently solvable in most practical cases.

⇒ Quite efficient in practice.

References I



J.-P. Katoen.

Reduced ordered binary decision diagrams, lecture #13 of advanced model checking, December 2010.